SAMS Kernel

The workbench: editing, assembling and running on the TI

Version 1.6.0 · 28 October 2026
For a TI-99/4A with an F18A video card and a SAMS memory card
MIT License, © 2026 Rebecca Sutter

Contents

1. Getting it onto the TI

2. The menu

2a. A worked example: the same program on both routes

3. The editor

4. The assembler

5. Writing a program for it

5a. On a TIPI

6. How it works, and what it costs

7. Checking it

The workbench is a program for the TI itself: an **editor**, an **assembler** and a **launcher** in one application, built on the kernel. With it you write a SAMS Kernel program, assemble it and run it on the TI, with no PC in the loop once the workbench is installed.

What is checked, and what is not. All of it runs in a simulator (a TMS9900, the F18A, SAMS, a keyboard, TI's own console ROM and disk controller ROM). The assembler's output is byte-identical to xas99's for every example module and for two probe sources that use every instruction in every operand form; the editor, the catalog, the assembling and the running are exercised by key scripts. **It has not run on a real TI-99/4A**, and I have not measured how long an assembly takes on one (part 6 gives instruction counts).

Routes. This document is the **[TI]** route. Each **[TI]** step has a **[PC]** counterpart:

[TI] in the workbench	[PC]
N, a file name; E to edit	any editor on a file
A assembles it	`python3 tools/skbuild.py APPDIR` (xas99)
R runs it	`python3 tools/sim9900.py build/APP/APP8.bin --keys ...`, js99er, or a real TI
the disk catalog (C)	your file manager; `xdm99.py disk.dsk` lists a disk image
`COPY "kernel.equ"` reads `KERNEL/EQU` on the disk	the build writes `kernel.equ`; `COPY "kernel.equ"` reads that

The same source assembles on both: it is [PC or TI] code, with the **[TI]** limits in part 4 and part 5.

Programs on the workbench disk (each assembles and runs on the TI; the tests do exactly that): `HELLO/A99` (keys echoed), `BEEP/A99` (notes on keys 1-8: `SNDTONE`), `MODE/A99` (Text 80 / Text 40: `VMODE`, `TXCOLS`), `PAGING/A99` (SAMS pages from the pool: `PINIT`, `PALLOC`, `MAPE`, `GETPB`/`SETPB`), `COPY/A99` (two files at once: `FSEL`; the sample `SRC` on the disk is what it copies), `JOY/A99` (both joysticks: `JOYGET`, `JOYKEY`), `CLOCK/A99` (the date and time from a TIPI's clock: `CLKGET`), `REVERS/A99` (Reversi: a whole program, 3,392 bytes, takes under a minute and a half to assemble at the simulator's estimate of the TI's speed; I have not timed it on a real TI), `MODES/A99` (fifteen display screens) and `PROBE1/A99` (an assembler test). Each is the example for one part of PROGRAMMERS-GUIDE.md (parts 1, 5, 6, 7, 9).

1. Getting it onto the TI

A PC is needed once, to make two things:

```
python3 tools/skbuild.py examples/workbench          build/workbench/workbench8.bin: the cartridge image (FinalGROM 99 "8"
format)
python3 tools/skdisk.py work.dsk --kernel-equ build/workbench/kernel.equ examples/hello/hello.a99 ...
                                                     a disk image holding KERNEL/EQU and your sources
```

`images/workbench8.bin` and `images/workbench.dsk` in the distribution are these, prebuilt (the disk holds KERNEL/EQU, the programs listed below, and a sample text file `SRC` for `COPY/A99`). Put the cartridge image where your cartridge device wants it, and the disk in DSK1 (a TIPI, a CF7+ or a floppy; how the image gets there depends on your hardware, and I have tried none of them). **KERNEL/EQU must come from the same build as the workbench cartridge**: it holds the addresses of the kernel's routines and equates in that build, and a program assembled against another build's addresses will not run.

The disk image is single-sided single-density (90K), the format TI's own controller and the simulator use. I have not tried it in js99er, whose disk controller setting matters (a double-density "Generic Disk" setting reads other images than TI's own); if it does not read it, make the disk in the format your setup wants.

You need the same machine as for any SAMS Kernel program: an F18A and 512K of SAMS.

What can be assembled here. KERNEL/EQU holds the kernel's symbols and the workbench's own (`PG_WB`, `PG_ASM`, `PGUSER`...). A source that needs equates only another application's build has cannot assemble: `examples/music` uses `PD_TUNE` and `PD_SCALE`, the pages its songs are built into, so it is not on the workbench disk (`beep` is the sound example there). `tests/test_kernel.py nativedisk` checks that everything on the disk assembles.

2. The menu

Key	
`N`	type a file name (ENTER done, FCTN S or FCTN 1 delete a character, FCTN 9 leaves). It starts as `DSK1.`

`C`	the catalog of the file name's disk: UP and DOWN pick, ENTER chooses, FCTN 9 returns
`E`	edit the file (a new file if it is not on the disk)
`A`	assemble the file; the result, or the errors with their line numbers, appear below
`R`	run the program just assembled
FCTN =	quit

TI file names have no dots, so a source file called `hello.a99` on a PC is `HELLO/A99` on the TI (xdm99 and `skdisk.py` do this): the name to type is `DSK1.HELLO/A99`.

2a. A worked example: the same program on both routes

The program (50 bytes once assembled, identical on both routes; it prints a line and does nothing else):

```
       COPY "kernel.equ"
       COPY "macros.a99"
       AORG >C000
       DATA INIT,TICK,INIT,0
INIT   .enter
       LI   R0,2*COLS+2
       LI   R1,MSG
       LI   R2,(RTEXT*16+RBG)*256
       BL   @PUTS
       .leave
       RT
TICK   RT
MSG    STRI 'Built on the TI'
```

[TI] In the workbench: **N**, type `DSK1.MINE/A99`, **ENTER**; **E**; type the program above, ENTER at the end of each line (indentation is spaces: the editor does not indent for you; a typing mistake is mended with CTRL H or FCTN 1, as the status bar shows); **CTRL A** saves it and assembles it (the menu says "Assembled: 50 bytes at >C000"; a mistake is listed with its line number: **E**, fix, CTRL A again); **R** runs it: "Built on the TI" appears on row 2 (on a TI the keyboard types capitals unless you lower ALPHA LOCK, so the line reads BUILT ON THE TI). `tests/test_kernel.py workbenchfull` does exactly these keystrokes.

[PC] Make a directory `mine/` holding the same text as `main.a99` and an `app.json`:

```
{"name": "MINE", "min_pages": 128, "modules": [{"name": "main", "source": "main.a99"}]}
```

then `python3 tools/skbuild.py mine` and `python3 tools/sim9900.py build/mine/mine8.bin --frames 100`: the screen prints with "Built on the TI" on row 2. The image `build/mine/mine8.bin` is a cartridge for js99er, a FinalGROM or a real TI, which the **[TI]** route cannot make.

3. The editor

A full-screen editor for DIS/VAR 80 files, up to 800 lines of 80 columns. The status bar shows the line, the column, INS or OVR, and `*` when there are unsaved changes.

Key	
arrows (FCTN E S D X)	move
FCTN 5 BEGIN, FCTN 6 PROC'D	start, end of the line
FCTN 7 AID, FCTN 8 REDO	up, down 20 lines
typing	inserts; FCTN 2 INS switches to overwriting
ENTER	splits the line at the cursor
CTRL H	deletes the character before the cursor; at the start of a line it joins the line to the one before (if both fit in 80 columns)
FCTN 1 DEL	deletes the character under the cursor
FCTN 3 ERASE	deletes the line

CTRL S	saves (trailing blanks cut)
CTRL A	saves and goes to the menu and assembles
CTRL X	back to the menu (press twice to leave unsaved changes)

The keys are those the kernel's keyboard layer decodes (`examples/diag` shows every chord). Lines are stored in SAMS pages in the order of a table, so inserting and deleting lines moves only the table.

4. The assembler

It reads xas99's dialect, the part the examples use:

- labels (at the start of a line, an optional colon), comments after the operand field, `*` or `;` lines;
- every TMS9900 instruction, with the operand forms `Rn`, `*Rn`, `*Rn+`, `@expr`, `@expr(Rn)`; `NOP`, `RT`; case does not matter (`mov r1,r2` is fine);
- `AORG`, `DATA`, `BYTE`, `TEXT`, `STRI`, `EVEN`, `EQU`, `COPY`, `END`, `BSS`; `LIST`, `UNL`, `TITL`, `IDT`, `PAGE` are accepted and ignored;
- the kernel's macros `.enter .leave .push R .pop R .intsvc .callmod page,entry`, built in; `COPY "macros.a99"` is accepted and ignored;
- expressions of `+ - * /` and parentheses, **evaluated strictly left to right** (as xas99 does: `2+3*4` is 20); decimal, `>hex`, `'c'` and `'cc'`, symbols, and `$` (the address where the statement starts; inside a `DATA` or `BYTE` list, the address of that item).

Instructions and `DATA` are placed on even addresses (a padding byte is added), and a label on such a line takes the aligned address, as in xas99.

symbol names	up to 12 characters, upper and lower case the same; at most about 580 symbols (the kernel's 274 included)
image	up to 4,096 bytes, in one run from the first `AORG` (gaps between `AORG`s are zero-filled)
`EQU`	its value must be defined already (xas99 has the same rule)
strings	`TEXT` and `STRI` take one quoted string, `''` for a quote; no empty `STRI ''`
`COPY "name"`	the file `name` on the main file's disk, dots turned into slashes (`kernel.equ` is `KERNEL/EQU`); up to three levels
not supported	user macros (`.defm`), `DEF`, `REF`, relocatable output, listings

Errors. The list shows the line number, a message, and `(in a COPYed file)` if the line is in an included file. Besides the errors xas99 reports, it also refuses a `SBO`/`SBZ`/`TB` displacement outside -128..127, a shift count over 15, an `XOP` level over 15, an `LDCR`/`STCR` count outside 1-16, a `BYTE` outside -128..255 and a name over 12 characters. Unknown operations, bad operands, undefined symbols, labels defined twice, jumps out of range or to odd addresses, `COPY` of a file that is not there, `AORG` that is not defined and a label whose value changes between the passes are errors, as in xas99.

One difference from xas99 is deliberate. xas99 assembles `LDCR Rn,16` as `>3401`, which is `STCR Rn,0`; the TI's own encoding for 16 bits is 0, `>3001`, and that is what the workbench produces.

5. Writing a program for it

A program is **one kernel module**, as in the programmer's guide, with these specifics:

```
       COPY "kernel.equ"            the kernel's routines and equates (the file KERNEL/EQU)
       COPY "macros.a99"            accepted, and the macros are built in
       AORG >C000                   a module lives at >C000
       DATA INIT,TICK,REDRAW,0      the entry table first
INIT   ...
```

- `R` puts the image into two SAMS pages, `PGUSER` (an equate in KERNEL/EQU: the workbench's first free page + 2), and switches to them with `GOMOD`; the program's `INIT` runs with R0 = the workbench's page.
- **To come back**: `LI R0,PG_WB / CLR R1 / BL @GOMOD`. The assembled image stays in the workbench's memory, so `R` runs it again without assembling. FCTN = quits the whole workbench.
- The program shares the kernel with the workbench. Free SAMS pages for your own use start at `PGUSER+2`; the editor's buffer uses the pages above that, and a program that uses them loses the editor's text (the file on the disk is safe).
- The program is not a cartridge: it exists in SAMS while the workbench runs. The workbench cannot write a cartridge image (a FinalGROM's storage is not something the TI can write); for that, build the program on the PC with `skbuild.py`.

5a. On a TIPI

[TI] The workbench works with whatever drives the TI has; a TIPI is one of them.

- **Where the files go.** Map `DSK1.` to a directory on the Pi that holds the workbench's files (the TIPI's own configuration does this), or use `PI.` paths in the file name (`DSK1.` and `PI.PROJ.HELLO/A99` both work; the field takes 31 characters, so map a drive if your paths are long). A TIPI keeps files in TIFILES format: **[PC]** `python3 tools/skdisk.py work.dsk --kernel-equ build/workbench/kernel.equ --dir tipi_files FILES...` writes the same files as TIFILES (`hello.a99.tfi`, ... named as xdm99 names them) for a TIPI directory; how your TIPI wants such a file named I have not checked, so rename them if it needs another form.
- **Includes.** `COPY "kernel.equ"` and the other includes are looked for beside the main file first (`PI.PROJ.KERNEL/EQU` for `PI.PROJ.HELLO/A99`), then on the device alone (`PI.KERNEL/EQU`, `DSK1.KERNEL/EQU`): one `KERNEL/EQU` at the top serves every project directory. KERNEL/EQU must still come from the same build as the workbench cartridge.
- **Speed and the catalog.** A TIPI asks the Pi for every record, so assembling there is slower than from RAM-like drives in ways I have not measured. The catalog (`C`) lists the drive in the file name's device; whether your TIPI answers the TI catalog request for a directory I have not tried.
- **The clock.** `programs/clock.a99` reads the TIPI's clock.
- **Tested** against a model of a TIPI in the simulator, not against a real one.

6. How it works, and what it costs

`asm.a99` is a module of 5.3K. It reads the source a line at a time with the kernel's file service, **twice** (pass 1 gives every label its address, pass 2 produces the bytes). The symbol table is a hash table of 14-byte entries in two SAMS pages mapped at `>E000`; the image is built in a 4K buffer in slot A. A `COPY` closes the file, assembles the included file, and reopens the first file, reading on to where it was (it could keep both open now that the file service has handles; it still closes and reopens, which is simpler and was tested first). Pass 2 does not read KERNEL/EQU again: its symbols stay in the table. The editor (`ed.a99`, 2.4K) keeps lines in SAMS pages (51 lines of 80 bytes a page) in the order of a table of slot numbers. The shell is `wb.a99` (2K).

Instructions executed, in the simulator, including reading the disk through TI's disk controller ROM and the 274-line KERNEL/EQU:

Source	Lines	Image	Instructions
`examples/hello`	67	194 bytes	1.9 million
`examples/diag`	630	2,260 bytes	6.8 million
`examples/modes`	754	2,870 bytes	8.2 million

A real TI's speed in seconds is not measured. As a guess only, at a few hundred thousand instructions a second, a small program would take seconds and the larger ones tens of seconds.

7. Checking it

```
python3 tests/test_kernel.py native          the probes byte for byte against xas99, the error file, LDCR
python3 tests/test_kernel.py nativemods      every example module assembled on the simulated TI, against xas99's
python3 tests/test_kernel.py workbench       the editor's keys, the catalog, a program returning to the workbench
python3 tests/test_kernel.py workbenchfull   type a program in the editor, assemble it, run it
```

They need the disk controller ROM (`TIFDC_ROM`, see `tools/tifdc.py`) and xdt99 (`XAS99`); without the ROM they say so and skip. `tools/asmharness.py` is the harness: `assemble(source, app)` runs the assembler in the simulator, `workbench_ui(keys, frames, files)` plays a key script against the whole workbench.

Not verified: a real machine, real timing, and the path from a TIPI, CF7+ or floppy to the workbench's disk.